

- The total number of points is 100.

Name: name

Student ID: number

1. In this problem, you will perform gradient descent on the function

$$f(x) = -2 \exp\{-(x_1 - 1)^2 - (x_2 - 1)^2\} - \exp\{-(x_1 + 2)^2 - (x_2 + 3)^2\},$$

where $x = [x_1, x_2]$ is a 2-d variable.

- (a) Write out the gradient $\nabla f(x)$ in analytical form.

- Let the component functions be:

$$g_1(x) = \exp\{-(x_1 - 1)^2 - (x_2 - 1)^2\}$$

$$g_2(x) = \exp\{-(x_1 + 2)^2 - (x_2 + 3)^2\}$$

- The partial derivatives are:

$$\frac{\partial f}{\partial x_1} = 4(x_1 - 1)g_1(x) + 2(x_1 + 2)g_2(x)$$

$$\frac{\partial f}{\partial x_2} = 4(x_2 - 1)g_1(x) + 2(x_2 + 3)g_2(x)$$

- Thus, the gradient is:

$$\nabla f(x) = \begin{bmatrix} 4(x_1 - 1)g_1(x) + 2(x_1 + 2)g_2(x) \\ 4(x_2 - 1)g_1(x) + 2(x_2 + 3)g_2(x) \end{bmatrix}$$

- (b) Using the software of your choice, write a function that takes a 2-d variable x and returns $f(x)$, as well as a function that takes a 2-d variable x and returns $\nabla f(x)$.

```
import numpy as numpyObject

def f(x):
    x1, x2 = x
    term1 = -2 * numpyObject.exp(-(x1 - 1)**2 - (x2 - 1)**2)
    term2 = -numpyObject.exp(-(x1 + 2)**2 - (x2 + 3)**2)
    return term1 + term2

def grad_f(x):
    x1, x2 = x
    g1 = numpyObject.exp(-(x1 - 1)**2 - (x2 - 1)**2)
    g2 = numpyObject.exp(-(x1 + 2)**2 - (x2 + 3)**2)
    df_dx1 = 4 * (x1 - 1) * g1 + 2 * (x1 + 2) * g2
    df_dx2 = 4 * (x2 - 1) * g1 + 2 * (x2 + 3) * g2
    return numpyObject.array([df_dx1, df_dx2])
```

- (c) Perform gradient descent with a software of your choice on f starting from two different values of x : $(0, 0)$ and $(-1, -1)$. Tune your step size α . You can tune by printing out how f changes during each step. The change should be gradual. What is a good step size for reasonably fast convergence? How many steps does convergence take? What minima did you find? (you should be able to identify the approximate local minima by just inspecting the formula of f).

- **Tuning the Step Size:** A step size of $\alpha = 0.15$ was found to be effective. Increasing the step size to $\alpha \geq 0.6$ causes the optimizer to oscillate or diverge from the narrow basins.
- **Convergence Speed:** In both trials, the function value $f(x)$ stabilizes significantly between steps 25 and 30, reaching high precision by step 50 for the $(0,0)$ trial and for $(-1,-1)$ it be around step 100.
- **Minima Identified:** The optimization results depend on the starting coordinates, as summarized below:

	Trial A: Start (0, 0)		Trial B: Start (-1, -1)	
Step	$f(x)$	Current Position	$f(x)$	Current Position
0	-0.7358	(0.00, 0.00)	-0.0367	(-1.00, -1.00)
25	-1.9998	(0.99, 0.99)	-0.6214	(-1.62, -2.25)
50	-2.0000	(1.00, 1.00)	-0.9765	(-1.95, -2.91)
75	-2.0000	(1.00, 1.00)	-0.9984	(-1.99, -2.99)
100	-2.0000	(1.00, 1.00)	-0.9999	(-2.00, -3.00)

- (d) Plot the paths of your gradient descent trajectories in the 2-d state space (need to save x locations along the path). Also plot the value of $f(x)$ on the y -axis vs. the number of steps on the x -axis.

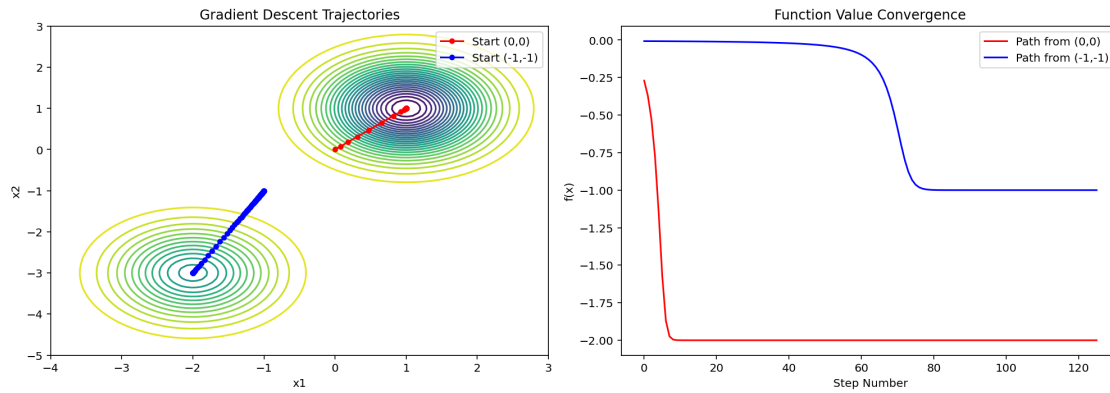


Figure 1: Gradient descent trajectories and function value convergence.